

Scoped Reproducibility Claims: A Workflow Verification Protocol for Experimental Research

Casper Marc Sluitman
College of Engineering
Shibaura Institute of Technology
3-7-5 Toyosu, Koto-ku, Tokyo, Japan
Casper.Sluitman@gmail.com

Shahrol Mohammadan
College of Engineering
Shibaura Institute of Technology
3-7-5 Toyosu, Koto-ku, Tokyo, Japan
mshahrol@shibaura-it.ac.jp

Abstract—Claims of reproducibility in experimental research are often ill-defined, and consequently fail to hold in practice, even when the code and data are published with the research. We therefore present a model for formally specifying reproducibility through *Scoped Reproducibility Claims (SRC)*, alongside a verification protocol for such claims. An SRC binds a target output and workflow to an explicit reproducibility boundary and provenance requirements to produce a qualified claim that can be evaluated by the same standards as other research claims. The protocol evaluates observed reproduction attempts against the SRC to produce attempt-level assessments grounded in provenance. To allow for general claims of reproducibility beyond any single observation, we introduce a complementary inference specification to compile multiple observations into a qualified general claim of reproducibility. We apply this model to an MNIST-QMNIST prediction-agreement analysis and show how this formal model can be integrated into computational research pipelines to automatically detect divergence between a workflow as executed and its reproducibility claims.

Keywords—computational reproducibility, replicability, provenance, research workflows, continuous integration, RO-Crate

I. INTRODUCTION

Publishing the code and data of a research workflow does not necessarily establish reproducibility. Systematic efforts to reproduce computational research frequently fail on the first attempt [1, 2, 3], although practices within software engineering show that software workflows can be made easy to consistently reproduce [4, 5]. This indicates an issue of protocol: research gets published without formal and verifiable claims of reproducibility, so there is limited incentive to ease the effort required to reproduce results. Additionally, without any such formal claims, the extent to which a workflow depends on external unreproducible inputs is unclear.

Therefore, we introduce a conceptual model for expressing claims of reproducibility as formal specifications, alongside a protocol for evaluating workflow executions against those specifications. The model represents attempt-level claims as *Scoped Reproducibility Claims (SRCs)* and generalisation claims through inference specifications over SRC assessment results, making reproducibility foundational to the overall research claim alongside any quantitative results. As an application of the model and protocol, we provide SRCs for

a MNIST-QMNIST prediction-agreement analysis [6], alongside supporting code and data [7] to reproduce our analysis results.

This paper thereby makes three novel contributions: (1) a conceptual model for expressing reproducibility claims as attempt-level and inference specifications, (2) a protocol for evaluating recorded workflow executions against these formal claims, and (3) a case study demonstrating and evaluating the model and protocol in practice; although the analysis in the case study is deliberately not novel, its contribution lies in the formal specification, verification, and assessment of its reproducibility.

II. BACKGROUND

A. Reproducibility and Replicability

Reproducibility and replicability are often used interchangeably, or otherwise inconsistently. Within this paper, we formally refer to a workflow as *reproducible* if its executions consistently produce the same outputs given the same inputs. For *replicability*, we are instead colloquially referring to research workflows that produce the same conclusions under theoretically grounded variability. While reproducibility is not necessarily a prerequisite for replicability, it does make replicability significantly easier to assess by precluding unaccounted variability. The protocol therefore supports both reproducibility and replicability in most definitions of the terms.

B. Related Work

To aid in computational reproducibility, the published code and data of a research workflow can be expanded into a research compendium, bundling all inputs required to reproduce the computational result [8]. Research compendia make the results more likely to be reproducible, though compendia do not necessarily make reproduction easy or consistent. This can be addressed with executable research compendia, containing programmed workflows alongside recoverable execution environments [9]. For some programming languages, the execution environment can be made sufficiently recoverable by pinning dependencies for its package manager. However, this often does not account for system-level variability without additional tooling. For example: while Python virtual environments

account for packaged dependencies, a system-level package manager is required to provide a compatible interpreter. The de facto standard within software engineering to account for most such variability are Docker containers [5], implementing the OCI container standard. These containers reduce variability by bundling code with the required runtime and system requirements, making them an ideal fit for executable research compendia.

Research compendia should record the provenance (origin, production, and use) of their artefacts. To account for this, we adopt the entity–activity–agent model defined by W3C PROV [10]. RO-Crate provides a specification for packaging research artefacts with metadata describing their identities, relationships, and provenance [11]; Workflow Run RO-Crate profiles specialise this model to describe workflow executions at different levels of granularity [12]. Similar to the continuous integration of software, automated workflow executions can build these packages and provide versioned assessments using regression tests or executable evaluations of any domain-specific criteria [13], [14].

Scoped Reproducibility Claims do not replace these established methodologies. Rather, SRCs allow for qualified claims of reproducibility within this ecosystem, or any other — while we use entity–activity–agent model vocabulary as common interface, SRCs can be implemented independently of the W3C PROV and RO-Crate standards.

III. METHODOLOGY

A. Conceptual Model

The verification protocol is a general procedure parameterised by a formally defined, domain-adapted SRC. It uses the SRC to determine what must be executed, which prerequisites are outside the reproducible scope, what constitutes valid evidence, and how that evidence is assessed. Consider the following unqualified natural-language claim:

The results of this study can be reproduced from the published data and code.

This claim is concise, but consequently unbounded and imprecise. A protocol for validating this claim may inadvertently depend on undocumented choices that require inspection of the implementation. The SRC specification instead makes these choices explicit by recording them as claim parameters interpreted by the protocol. Moreover, the unqualified claim presents reproducibility as a general property of the workflow without specifying an inferential basis. To account for this, the protocol associates each SRC evaluation result with its configurations and assesses the generalisation according to an inference specification. These specifications are defined as follows:

$$\text{SRC} = \langle X, W, B, \mathcal{V} \rangle, \quad \mathcal{V} = \langle \mathcal{P}, \mathcal{A}, \delta \rangle, \quad (1)$$

$$\mathcal{I} = \langle C, R, M, g \rangle. \quad (2)$$

SRC is the claim specification for an individual reproduction attempt, where:

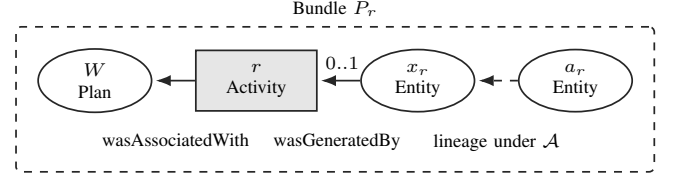


Fig. 1. Representation of an individual reproduction attempt following W3C PROV-DM [10]: provenance Bundle P_r records reproduction Activity r , which is associated with workflow Plan W , and when produced, output instance Entity x_r . Under contract $\mathcal{A}$, a dependency through $\mathcal{V}$ in SRC, evidence Entity a_r must be attributable to r , and where x_r exists, trace its lineage to x_r .

- X is the research output being reproduced;
- W is the workflow producing output X ;
- B is the boundary specification, defining what must be reproduced within the scope and what must be provided from outside it; and
- $\mathcal{V}$ is the attempt-assessment specification, wherein:
 - $\mathcal{P}$ specifies the provenance record required to verify an attempted execution of workflow W under boundary specification B , including how the attempt ended and whether it conformed to B ;
 - $\mathcal{A}$ is the contract that determines whether evidence from an attempt is eligible for assessing the reproduction of output X ; and
 - δ is the decision rule applied to evidence eligible under contract $\mathcal{A}$ to decide whether output X was reproduced.

$\mathcal{I}$ is the inference specification for determining whether multiple SRC evaluation results support a qualified general claim, where:

- C determines whether an SRC evaluation occurred under conditions covered by the general claim;
- R defines how attempts sample those contexts;
- M defines how excluded, inadmissible, or otherwise unassessable attempts are accounted for; and
- g is the rule applied to the evaluation results to determine whether they support the general claim.

When assessing a qualified general claim, the protocol first evaluates individual reproduction attempts against the SRC and applies inference specification $\mathcal{I}$ to the combined assessment results. Fig. 1 represents an individual attempt. For attempt r , provenance P_r records an attempted execution of workflow W and its termination state. When the attempt produces output instance x_r , P_r records its generation. Under contract $\mathcal{A}$, evidence a_r must be attributable to r , and where x_r exists, traceable to x_r . Operation VERIFY evaluates P_r and a_r against SRC to produce assessment result y_r :

$$y_r = \text{VERIFY}(\text{SRC}, P_r, a_r). \quad (3)$$

Result y_r classifies the result of an attempt, so that only an admissible attempt is judged by δ . For each attempt r , the protocol derives scope realisation b_r and context realisation c_r from provenance P_r :

$$P_r \mapsto \begin{cases} b_r = \text{SCOPE}(P_r, B), \\ c_r = \text{CONTEXT}(P_r, C). \end{cases} \quad (4)$$

Boundary specification B determines the reproducible scope of an SRC, as realised in b_r . Each b_r describes only one attempt and consequently cannot alone support a claim generalising across conditions. A general claim instead requires multiple evaluation results assessed under an inference specification $\mathcal{I}$. Fig. 2 classifies experimental designs by whether b_r and c_r are fixed or varied. A variable inside B contributes to b_r ; a variable inside in P_r but outside B contributes to c_r when it is interpreted by C . When b_r is fixed, attempts share the same scope realisation; when c_r is fixed, they share the same recorded context. Claims dependent on variables not recorded in provenance P_r are beyond this model, since they are not recorded and can therefore not be assessed.

Fig. 3 shows how an arbitrary attempt r relates to workflow conformance and execution context through its realised scope b_r and context c_r . Suppose an SRC concerns a reaction sensitive to temperature, with reaction temperature inside B and ambient temperature outside B but recorded in P_r . Boundary specification B prescribes the range of reaction temperatures conformant to the workflow and is therefore a constraint on individual attempts. Context specification C defines the ambient-temperature ranges over which $\mathcal{I}$ assesses the general claim, defined to account for ambient temperature as confounder in attributing production of X to the workflow. Both specifications are therefore constraints on the assessment, but B applies to individual attempts whereas C applies to inference across those results.

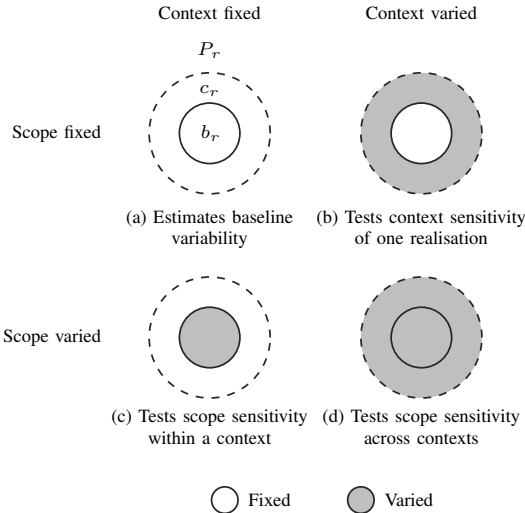


Fig. 2. Experimental designs for repeated workflow assessment, where each attempt r is recorded in provenance P_r . The inner circle represents scope realisation b_r , while the outer ring represents context realisation c_r . The shaded regions indicate which realisations vary between attempts, while unshaded regions are fixed.

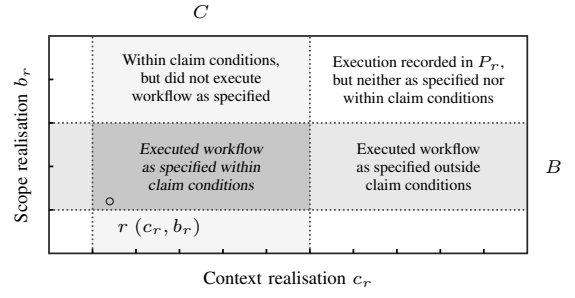


Fig. 3. Relationship between the scope and context realisations of attempt r : the band labelled B consists of scope realisations conformant to the workflow, while the band labelled C consists of context realisations under $\mathcal{I}$. The bands represent any acceptance criterion, such as measurement thresholds or categorical constraints. Their overlap satisfies both conditions, wherein the dot labelled $r(c_r, b_r)$ represents an attempt conforming to the workflow within the claim conditions.

B. Verification Protocol

The protocol is a reference process for assessing the reproducibility of experimental research, expressed through the conceptual model. While it is designed to support computational workflows specifically, the protocol can be applied to any workflow execution for which the required provenance and evidence are recorded. This includes the manual enactment of physical laboratory workflows, though strict controls over context and sampling may be difficult to implement outside a digital environment. The protocol is divided into four stages subdivided into steps, corresponding to Fig. 4:

Stage 1 — Bind applicable specifications

- (a) For a general claim of reproducibility, set inference selection $\mathcal{J} = \mathcal{I}$. For an attempt-level claim, set $\mathcal{J} = \emptyset$. Implement and apply BIND (Eq. (5)) to resolve the dependencies required by the SRC and $\mathcal{J}$ and produce binding record κ .
- (b) Check binding record κ for completeness; if incomplete, return it as an input-level result to record the failure separately from any assessed attempts.

By resolving dependencies before any attempt is assessed, binding record κ separates missing inputs from failed reproduction:

$$\kappa = \text{BIND}(\text{SRC}, \mathcal{J}), \quad \mathcal{J} \in \{\emptyset, \mathcal{I}\}. \quad (5)$$

These procedures specify the input–output mappings that must be implemented to apply the protocol, but the protocol itself does not prescribe any implementation for these mappings. It instead provides shared vocabulary for describing concrete implementations, as shown the case study.

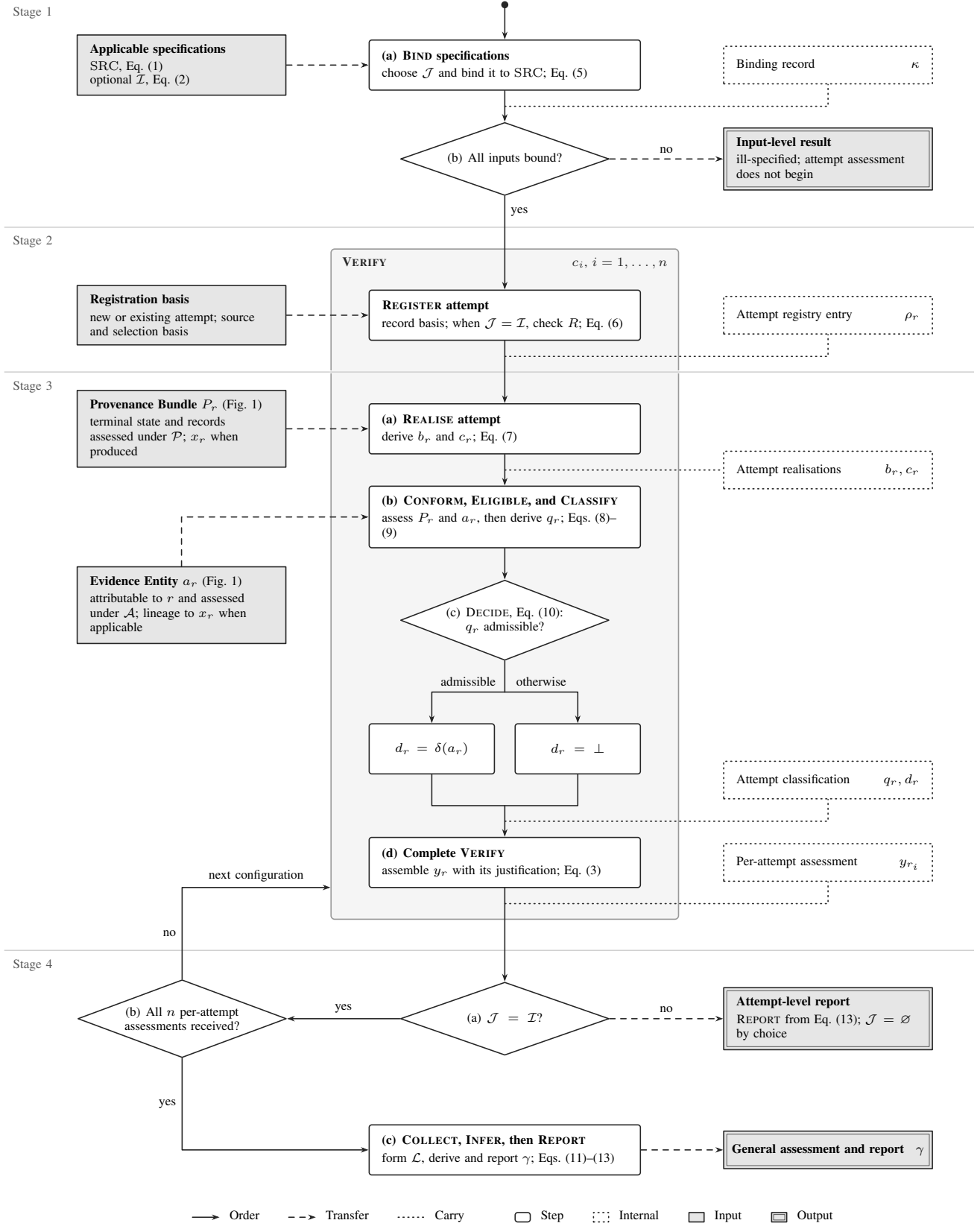


Fig. 4. Diagram of verification protocol for assessing reproduction attempts against SRC. Inputs flow from the left into protocol steps, with artefacts produced by these steps shown on the right (both internal resources and protocol outputs). stage 1 binds and checks the specifications, stage 2 registers each attempt across execution contexts, stage 3 produces a per-attempt assessment, and stage 4 collects these assessments for inference based on a bound $\mathcal{I}$ to support a general claim of reproducibility.

Stage 2 — Register an attempt

Implement and apply REGISTER (Eq. (6)) to create attempt record ρ_r . Source description source_r must identify the origin of provenance record P_r , while selection description selection_r must explain how attempt r was selected for assessment. When $\mathcal{J} = \mathcal{I}$, also indicate whether the selection conforms to R .

The source description source_r identifies whether the provenance and evidence were obtained directly from a recently terminated execution or retrieved from a preserved record. Attempt record ρ_r compiles this origin with the selection description selection_r :

$$\rho_r = \text{REGISTER}(r, \text{source}_r, \text{selection}_r; \mathcal{J}). \quad (6)$$

Registering attempt r before reproduction decision d_r is resolved in step 3(c) prevents that decision from influencing whether the attempt is included. Attempt record ρ_r preserves the condition for the inclusion, so that it can be checked against R when $\mathcal{J} = \mathcal{I}$.

Stage 3 — Verify one attempt

- (a) Implement and apply REALISE (Eq. (7)) to derive attempt realisations b_r, c_r from provenance record P_r .
- (b) Implement and apply CONFORM and ELIGIBLE (Eq. (8)) to determine conformance with B and evidence eligibility, respectively. Then implement and apply CLASSIFY (Eq. (9)) to combine both into attempt classification q_r .
- (c) Implement and apply DECIDE (Eq. (10)) to derive reproduction decision d_r .
- (d) Implement and apply VERIFY (Eq. (3)) to assemble per-attempt assessment y_r and its justification.

This stage is central to the protocol and involves some control flow, so Stage 3 and Fig. 4 present it differently according to their respective purposes. Whereas the enumerated steps decompose the required operations into individually executable procedures, the diagram shows how they compose within the overall protocol flow. Accordingly, VERIFY encompasses the complete production of assessment y_r in the diagram, while step 3(d) represents only the assembly of its results.

Procedure REALISE interprets provenance record P_r under boundary specification B and context specification C , producing scope realisation b_r and context realisation c_r for the attempt r :

$$\begin{aligned} \langle b_r, c_r \rangle &= \text{REALISE}(P_r; B, C) \\ &= \langle \text{SCOPE}(P_r, B), \text{CONTEXT}(P_r, C) \rangle. \end{aligned} \quad (7)$$

Procedures CONFORM and ELIGIBLE operationalise provenance specification $\mathcal{P}$ and evidence contract $\mathcal{A}$ from the conceptual model, producing conformance value χ_r and eligibility value ϵ_r :

$$\begin{aligned} \chi_r &= \text{CONFORM}(P_r; \mathcal{P}, B), \\ \epsilon_r &= \text{ELIGIBLE}(a_r; \mathcal{A}, P_r). \end{aligned} \quad (8)$$

Conformance value χ_r is conformant, nonconformant, or undetermined; eligibility value ϵ_r is eligible, ineligible, or undetermined. Procedure CLASSIFY compiles these into attempt classification q_r . The attempt is admissible if both checks pass, inadmissible if either check fails, and otherwise unassessable:

$$q_r = \text{CLASSIFY}(\chi_r, \epsilon_r). \quad (9)$$

The resulting attempt classification q_r determines whether DECIDE may apply decision rule δ to evidence a_r . It produces reproduction decision $d_r = \delta(a_r)$ only for an admissible attempt; for any other classification, $d_r = \perp$ remains undefined:

$$\begin{aligned} d_r &= \text{DECIDE}(q_r, a_r; \delta) \\ &= \begin{cases} \delta(a_r), & q_r = \text{admissible}, \\ \perp, & \text{otherwise}, \end{cases} \end{aligned} \quad (10)$$

Procedure VERIFY thereafter assembles these results and their supporting records into per-attempt assessment y_r , as defined in Eq. (3).

Stage 4 — Collect, infer, and report

- (a) If $\mathcal{J} = \emptyset$, implement and apply REPORT (Eq. (13)) to return an attempt-level report based on the lone per-attempt assessment y_r .
- (b) Otherwise, check whether all n required per-attempt assessments y_{r_i} have been received; if not, repeat stages 2 and 3.
- (c) Implement and apply COLLECT (Eq. (11)) to form assessment collection $\mathcal{L}$, INFER (Eq. (12)) to derive general assessment γ , and REPORT (Eq. (13)) to return the general report.

This stage branches based on inference selection $\mathcal{J}$, as is shown more explicitly in Fig. 4: when $\mathcal{J} = \emptyset$, the protocol returns an attempt-level report without compiling assessments; when $\mathcal{J} = \mathcal{I}$, procedure COLLECT compiles the per-attempt assessments required by R into assessment collection $\mathcal{L}$:

$$\mathcal{L} = \text{COLLECT}(R, \langle y_{r_i} \rangle_{i=1}^n). \quad (11)$$

Procedure INFER applies M to account for excluded, inadmissible, or otherwise unassessable attempts in assessment collection $\mathcal{L}$ before g determines general assessment γ :

$$\gamma = \text{INFER}(\mathcal{I}, \mathcal{L}) = g(M(\mathcal{L}; C, R)). \quad (12)$$

Procedure REPORT returns the applicable report: when $\mathcal{J} = \emptyset$, it reports per-attempt assessment y_r without inference;

when $\mathcal{J} = \mathcal{I}$, it reports general assessment γ with assessment collection $\mathcal{L}$:

$$\begin{aligned} \text{REPORT}(\emptyset, y_r) &\longrightarrow \text{attempt-level report,} \\ \text{REPORT}(\mathcal{I}, \mathcal{L}, \gamma) &\longrightarrow \text{general report.} \end{aligned} \quad (13)$$

As with every component of this protocol, the representation of a report is decided by the implementation. Within the explored ecosystem, it may be represented comprehensibly as an RO-Crate, as a qualified natural-language claim linked to its supporting records, or any other qualified representation.

C. Case-Study

The MNIST handwritten-digit dataset was created by processing an established NIST dataset into consistently sized greyscale images, but the exact preprocessing procedure has been lost to time. The QMNIST dataset was created by applying a reconstruction of that procedure to the original NIST source images [6]. We evaluate whether this reconstruction preserves the predictions of a small pre-trained MNIST classifier [15]. This leaves model training out of the reproducible boundary, so that reproduction only requires inference and analysis rather than training, which is possible across standard hosted CI/CD runners for a model this small. As shown in

Fig. 5, GitHub Actions automatically evaluates our claims across environments and fails the pipeline whenever a claim is no longer valid as defined.

Scoped Reproducibility Claim. *There is at least 99% agreement between predictions of the classifier for MNIST and QMNIST sample pairs. This can be reproduced from the published data, model, and code across the containerised and uncontainerised environments evaluated here.*

The pipeline first publishes a Docker image to the container registry under an immutable digest, providing reproductions with an exact copy of the execution environment under B . When all claims succeed, the manuscript is compiled from its $\text{\LaTeX}$ source inside this image and the PDF is published as a GitHub Release. To account for variability outside B , the pipeline evaluates the claims across three containerised and three uncontainerised environments, recording each attempt using the Workflow Run RO-Crate profiles. These records are published alongside the PDF to provide provenance.

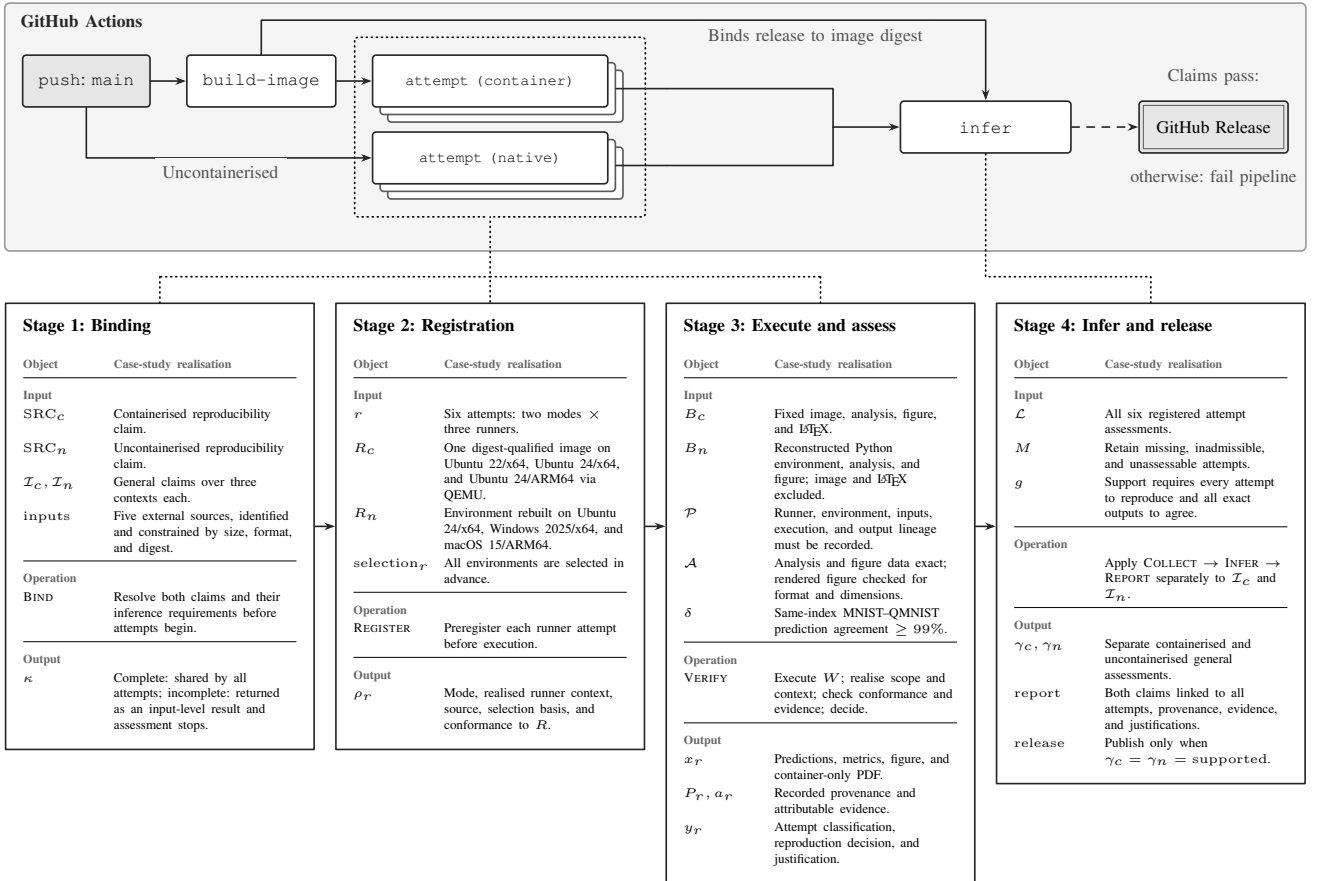


Fig. 5. For every code change pushed to the `main` branch, the SRCs are automatically verified by the supporting software [7]. The GitHub Actions diagram above shows the automated execution, while the columns below show how stages of the verification protocol are realised. The code is executed on Linux, Windows and macOS; and across three Linux runner contexts inside containers instantiated from an archived image.

IV. RESULTS

As shown in Fig. 6, across the 10,000 MNIST-QMNIST image pairs, the MNIST-12 prediction differed only three times between the original MNIST digit and the QMNIST reconstruction. This supports the prediction-agreement component of the SRC with a wide margin.

This was reflected in the recorded assessments: all six registered attempts were admissible and met the registered criteria for a successful reproduction. Provenance shows that all three containerised attempts executed within the same image, identified by their digest, while the three uncontainerised attempts successfully recovered the Python environment on Ubuntu, Windows, and macOS. Every artefact resolved to the same digest across its three contexts, and every Workflow Run RO-Crate passed validation.

Anecdotally, the pipeline caught several cases in which a code change made an attempt inadmissible or changed a registered output. Since this document reports those claims, these failures could have hypothetically gone unnoticed and found themselves published. We received an automatic e-mail after each such commit to address the regression. This behaviour is consistent with CI/CD pipelines commonplace within software engineering, and is therefore not specific to GitHub as a platform.

QMNIST prediction-agreement criterion

MNIST-12 predictions for 10,000 same-index MNIST/QMNIST pairs

Observed agreement		Registered minimum
99.97%	$\geq$	99.00%
9997 / 10 000 pairs		9900 / 10 000 pairs
Criterion met by 97 pairs (0.97 pp)		

Changed prediction pairs

MNIST-12 predictions for the three changed MNIST/QMNIST pairs

	MNIST	Δ	QMNIST	
Pair 92 True label = 9				4 → 9 wrong to correct
Pair 619 True label = 1				8 → 1 wrong to correct
Pair 4265 True label = 4				8 → 9 wrong to wrong

Fig. 6. MNIST-12 prediction agreement for 10,000 MNIST-QMNIST pairs, showing the three pairs for which predictions differed. Pixel differences marked with * are only 1/255, so they are shown at full contrast.

V. CONCLUSION

Scoped Reproducibility Claims show that qualifying claims of reproducibility allows them to be evaluated by the same standards as other research claims. Such claims can be integrated into both programmatic and physical workflows, the former of which is exemplified by our QMNIST case study; where, in practice, automatically evaluating the SRC helped keep the implementation and reported claims aligned throughout development. However, while the concept is demonstrably practical, especially when applied to programmable pipelines, the amount of syntax required to specify the protocol may hinder adoption in favour of more flexible methodologies. Future work could therefore focus on the engineering challenges of developing high-level, domain-specific implementations of these concepts to preserve the coverage of the formally defined protocol, while reducing the specification burden.

REFERENCES

- [1] *Reproducibility and replicability in science*, Sep. 2019. DOI: 10.17226/25303. [Online]. Available: <http://dx.doi.org/10.17226/25303>.
- [2] Y.-M. Kim, J.-B. Poline, and G. Dumas, “Experimenting with reproducibility: A case study of robustness in bioinformatics,” *GigaScience*, vol. 7, no. 7, giy077, Jul. 2018, ISSN: 2047-217X. DOI: 10.1093/gigascience/giy077. eprint: <https://academic.oup.com/gigascience/article-pdf/7/7/giy077/25178539/giy077.pdf>. [Online]. Available: <https://doi.org/10.1093/gigascience/giy077>.
- [3] S. Samuel and D. Mitchen, “Computational reproducibility of jupyter notebooks from biomedical publications,” *GigaScience*, vol. 13, giad113, Jan. 2024, ISSN: 2047-217X. DOI: 10.1093/gigascience/giad113. eprint: <https://academic.oup.com/gigascience/article-pdf/doi/10.1093/gigascience/giad113/60708559/giad113.pdf>. [Online]. Available: <https://doi.org/10.1093/gigascience/giad113>.
- [4] C. Lamb and S. Zacchiroli, “Reproducible builds: Increasing the integrity of software supply chains,” *IEEE Software*, vol. 39, no. 2, pp. 62–70, Mar. 2022. DOI: 10.1109/ms.2021.3073045. [Online]. Available: <http://dx.doi.org/10.1109/MS.2021.3073045>.
- [5] C. Boettiger, “An introduction to docker for reproducible research,” en, *ACM SIGOPS Operating Systems Review*, vol. 49, no. 1, pp. 71–79, Jan. 2015. DOI: 10.1145/2723872.2723882. [Online]. Available: <http://dx.doi.org/10.1145/2723872.2723882>.
- [6] C. Yadav and L. Bottou, “Cold case: The lost mnist digits,” in *Advances in Neural Information Processing Systems*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d’Alché-Buc, E. Fox, and R. Garnett, Eds., vol. 32, Curran Associates, Inc., 2019. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2019/file/51c68dc084cb0b8467eafad1330bce66-Paper.pdf.

- [7] C. M. Sluitman and S. Mohammadan, *Executable QM-NIST case study for scoped reproducibility claims*, version 0.1.0, Source code and supporting data, 2026. Accessed: Aug. 17, 2026. [Online]. Available: <https://surl.jp/Q7uK6heC>.
- [8] R. Gentleman and D. T. Lang, "Statistical analyses and reproducible research," *Journal of Computational and Graphical Statistics*, vol. 16, no. 1, pp. 1–23, 2007. DOI: 10.1198/106186007X178663. eprint: <https://doi.org/10.1198/106186007X178663>. [Online]. Available: <https://doi.org/10.1198/106186007X178663>.
- [9] D. Nüst et al., "Opening the publication process with executable research compendia," *D-Lib Magazine*, vol. 23, no. 1/2, Jan. 2017. DOI: 10.1045/january2017-nuest. [Online]. Available: <https://doi.org/10.1045/january2017-nuest>.
- [10] "PROV-DM: The PROV data model," World Wide Web Consortium, W3C Recommendation, Apr. 30, 2013. [Online]. Available: <https://www.w3.org/TR/2013/REC-prov-dm-20130430/>.
- [11] S. Soiland-Reyes et al., "Packaging research artefacts with ro-crate," *Data Science*, vol. 5, pp. 1–42, Jan. 2022. DOI: 10.3233/DS-210053.
- [12] S. Leo et al., "Recording provenance of workflow runs with ro-crate," *PLOS ONE*, vol. 19, no. 9, pp. 1–35, Sep. 2024. DOI: 10.1371/journal.pone.0309210. [Online]. Available: <https://doi.org/10.1371/journal.pone.0309210>.
- [13] G. K. Sandve, A. Nekrutenko, J. Taylor, and E. Hovig, "Ten simple rules for reproducible computational research," *PLOS Computational Biology*, vol. 9, no. 10, pp. 1–4, Oct. 2013. DOI: 10.1371/journal.pcbi.1003285. [Online]. Available: <https://doi.org/10.1371/journal.pcbi.1003285>.
- [14] G. Wilson et al., "Best practices for scientific computing," *PLOS Biology*, vol. 12, no. 1, pp. 1–7, Jan. 2014. DOI: 10.1371/journal.pbio.1001745.
- [15] ONNX Model Zoo, *MNIST-12: Handwritten digit recognition*, version ONNX opset 12, Pretrained convolutional MNIST classifier, 2022. Accessed: Jul. 22, 2026. [Online]. Available: <https://github.com/onnx/models/tree/4c46cd00fbd7cd30b6c1c17ab54f2e1f4f7b177/validated/vision/classification/mnist>.